Message Queue Lab Report — Six Exercises on RabbitMQ and Kafka

2110521 Software Architecture — Individual Assignment 4
Student: **Watayut Aiamanan** · Student ID: **6970267821** · Submitted: 13 October 2026
Brokers: RabbitMQ 4.3.6 (management UI) and Apache Kafka 4.1.2 (KRaft) from the lab `compose.yaml`, both `healthy` · Clients: Python 3.13, pika 1.3.2, confluent-kafka 2.11.1 · Evidence: terminal captures, RabbitMQ management-UI screenshots (Playwright), Kafka JSONL logs in `evidence/`

Exercise 1 — RabbitMQ: Crash Before ACK

Scenario. A worker consumes one message, commits a row to SQLite, and then dies with `os._exit(99)` *before* sending `basic_ack`. The worker is then restarted without publishing again. The experiment is run in *unsafe* mode (no deduplication) and in *safe* mode (an `INSERT OR IGNORE` dedupe marker in the same transaction as the business effect).

Unsafe mode — terminal output

```
ex1-unsafe-1-publish — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py publish --mode unsafe --run unsafe5
2026-10-13T14:37:58.188+00:00 CONFIRMED id=ee756d9b-8b89-4c13-9825-88f4cd1ea838 payload={'action': 'create_order'}
```

Publish one message to the unsafe queue (publisher confirm received)

```
ex1-unsafe-2-worker-crash — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py worker --mode unsafe --run unsafe5 --crash-before-ack
2026-10-13T14:38:05.413+00:00 READY queue=lab.failure.crash.unsafe.unsafe5; processes one delivery then exits
2026-10-13T14:38:05.413+00:00 RECEIVED id=ee756d9b-8b89-4c13-9825-88f4cd1ea838 tag=1 redelivered=False
2026-10-13T14:38:05.421+00:00 DB COMMITTED: applied=True, business_effect_count=1
2026-10-13T14:38:05.421+00:00 INTENTIONAL CRASH: database committed, ACK NOT SENT (exit code 99)
[process exited with code 99]
```

First worker run: DB committed with business_effect_count=1, then the intentional crash before ACK (exit 99)

```
ex1-unsafe-3-worker-restart — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py worker --mode unsafe --run unsafe5
2026-10-13T14:38:08.189+00:00 READY queue=lab.failure.crash.unsafe.unsafe5; processes one delivery then exits
2026-10-13T14:38:08.189+00:00 RECEIVED id=ee756d9b-8b89-4c13-9825-88f4cd1ea838 tag=1 redelivered=True
2026-10-13T14:38:08.194+00:00 DB COMMITTED: applied=True, business_effect_count=2
2026-10-13T14:38:08.194+00:00 ACK SENT
```

Restarted worker: the same message id comes back with redelivered=True and is applied a second time, business_effect_count=2

Safe mode — terminal output

```
ex1-safe-1-publish — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py publish --mode safe --run safe5
2026-10-13T14:38:11.014+00:00 CONFIRMED id=a9af4c81-27d4-4ecc-9663-0ff0d719915e payload={'action': 'create_order'}
```

Publish one message to the safe queue

```
ex1-safe-2-worker-crash — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py worker --mode safe --run safe5 --crash-before-ack
2026-10-13T14:38:16.904+00:00 READY queue=lab.failure.crash.safe.safe5; processes one delivery then exits
2026-10-13T14:38:16.904+00:00 RECEIVED id=a9af4c81-27d4-4ecc-9663-0ff0d719915e tag=1 redelivered=False
2026-10-13T14:38:16.910+00:00 DB COMMITTED: applied=True, business_effect_count=1
2026-10-13T14:38:16.910+00:00 INTENTIONAL CRASH: database committed, ACK NOT SENT (exit code 99)
[process exited with code 99]
```

First safe worker run: applied=True, business_effect_count=1, then the crash before ACK

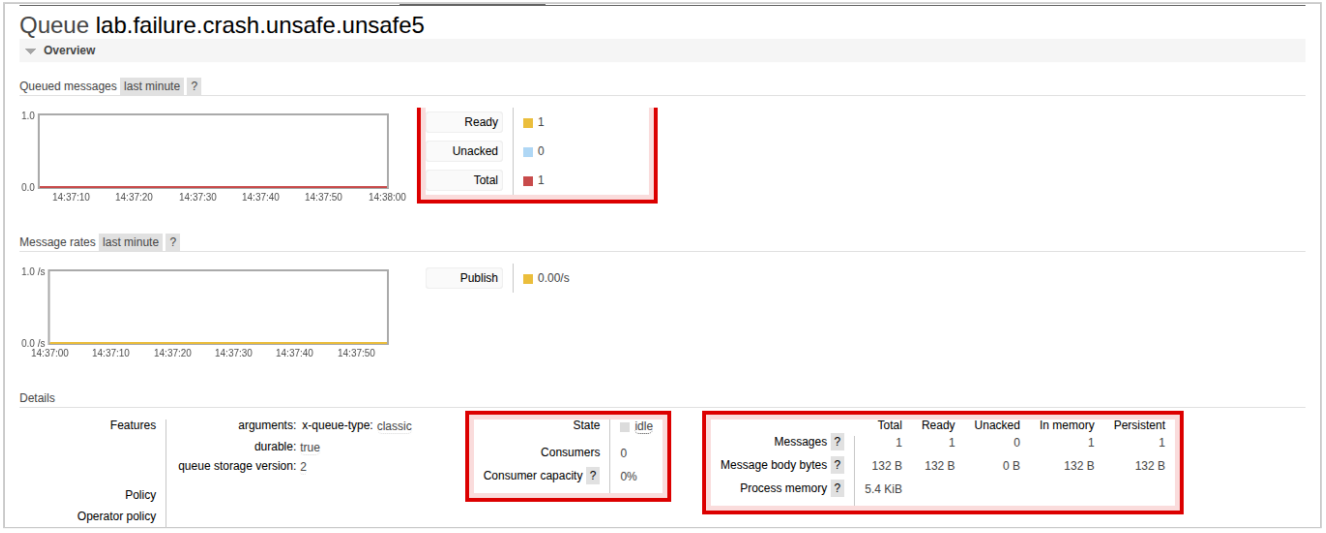
```
ex1-safe-3-worker-restart — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo1_crash_ack.py worker --mode safe --run safe5
2026-10-13T14:38:19.721+00:00 READY queue=lab.failure.crash.safe.safe5; processes one delivery then exits
2026-10-13T14:38:19.721+00:00 RECEIVED id=a9af4c81-27d4-4ecc-9663-0ff0d719915e tag=1 redelivered=True
2026-10-13T14:38:19.722+00:00 DB COMMITTED: applied=False, business_effect_count=1
2026-10-13T14:38:19.722+00:00 ACK SENT
```

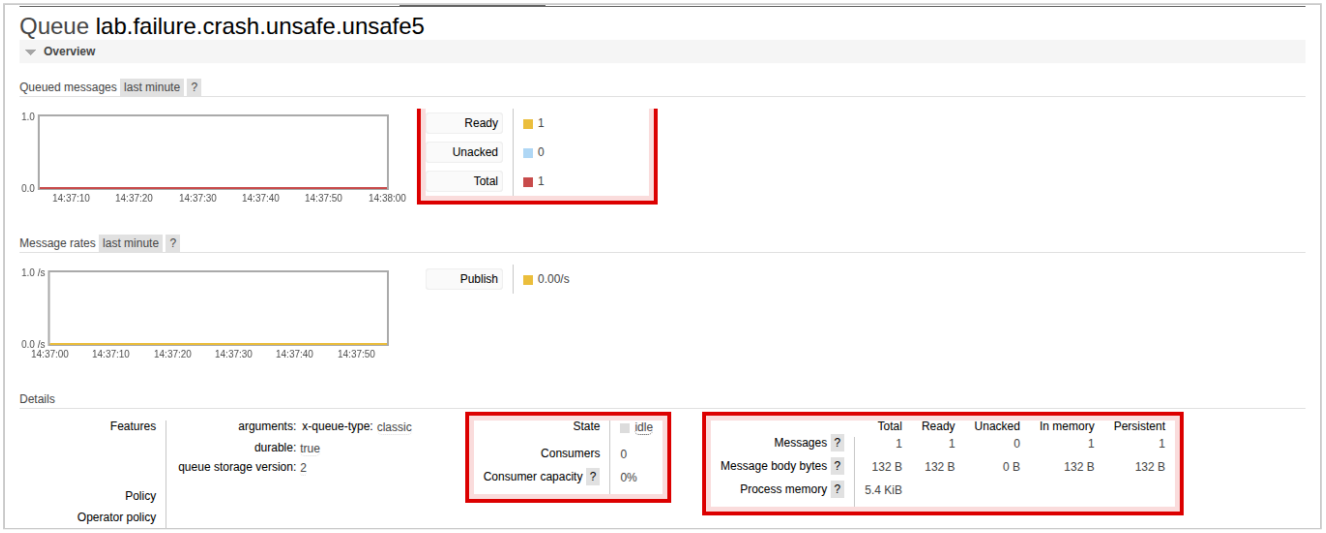
Restarted safe worker: redelivered=True but applied=False and business_effect_count stays 1

RabbitMQ management UI

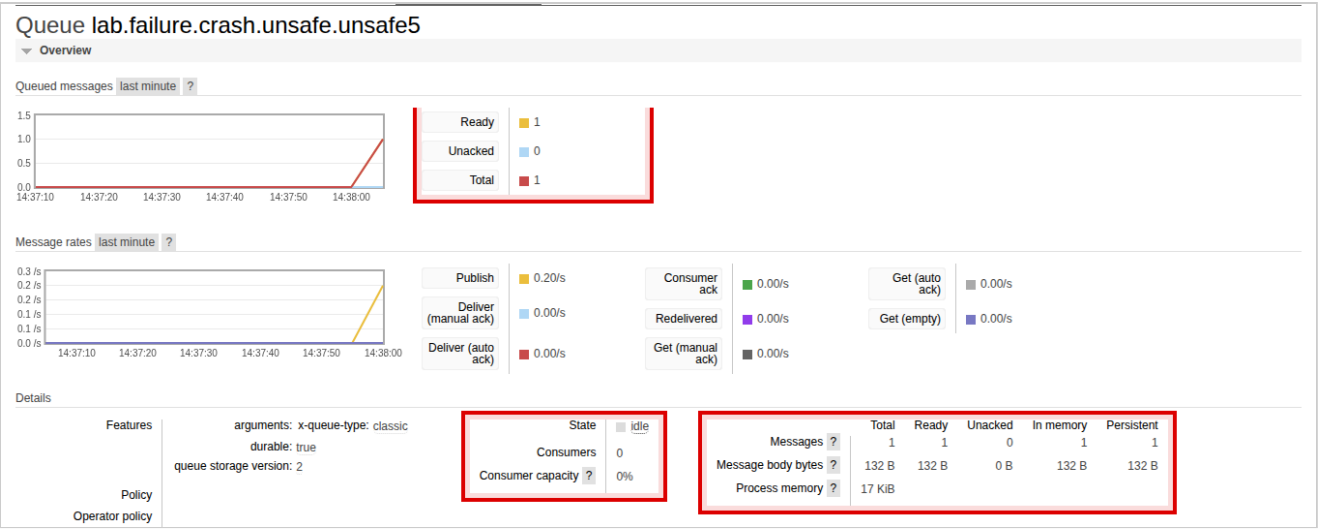
The management-UI screenshots are cropped to the relevant part of the page and the values that matter are outlined in red.



After publish: 1 message Ready, 0 Unacked, 0 consumers, message persistent on a durable queue



Immediately after the worker crashed: the message is back to Ready=1 because the broker never received the ACK and the consumer's connection closed



After the restarted worker ACKed: the queue is empty (Ready=0, Total=0)

Results

Run	`redelivered`	`applied`	`business_effect_count`
Unsafe — first run (crash)	False	`True`	`1`
Unsafe — restart	**True**	`True`	**`2`** ← duplicate
Safe — first run (crash)	False	`True`	`1`
Safe — restart	**True**	**`False`**	**`1`** ← deduplicated

Explanation

RabbitMQ delivers at-least-once: a message delivered to a consumer stays *unacknowledged* until the consumer sends `basic_ack`, and when that consumer's connection dies the broker puts the message back on the queue with the `redelivered` flag set, which is exactly what the UI screenshots show (Ready went 1 → 0 → 1 → 0). In unsafe mode the worker has no memory that it already handled event `ee756d9b…`, so the redelivery is treated as a brand-new order and a second row is inserted, giving `business_effect_count=2`. In safe mode the worker first executes `INSERT OR IGNORE INTO processed(event_id)`; on the redelivery the row already exists, `rowcount` is 0, the business insert is skipped and the count stays at 1, after which the worker ACKs so the message is finally removed. The single transaction is essential: because the dedupe marker and the business row are committed together, there is no window in which the effect is committed but the marker is not (or vice versa), so a crash at any point leaves the database either fully "done" or fully "not done". Using `os._exit(99)` rather than `sys.exit()` matters because `sys.exit` raises `SystemExit`, which would let pika's `finally` blocks close the channel gracefully, whereas `os._exit` kills the process instantly like a real crash and leaves the message unacknowledged.

Exercise 2 — RabbitMQ: Late Subscribers

Scenario. A durable fanout exchange is created. `setup` declares and binds a durable queue *before* anything is published. The `BEFORE` message is published while no consumer is running. Then a durable consumer (attaching to the pre-existing queue) and a temporary consumer (declaring a new exclusive auto-delete queue and binding it *now*) are started, and `AFTER` is published.

Terminal output

ex2-1-setup — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo2_late_subscriber.py setup --run fanout5
2026-10-13T14:38:22.525+00:00 SETUP: durable queue lab.failure.fanout.fanout5.saved exists and is bound; no consumer yet
```

Setup: durable queue declared and bound to the fanout exchange, no consumer yet

ex2-2-publish-before — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo2_late_subscriber.py publish --run fanout5 --message BEFORE
2026-10-13T14:38:22.666+00:00 CONFIRMED id=7f036349-039f-46a9-a3c2-bfb898546490 payload={'text': 'BEFORE'}
```

Publish BEFORE while no consumer is running

ex2-3-durable-consumer — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo2_late_subscriber.py durable --run fanout5
2026-10-13T14:38:28.618+00:00 READY durable queue=lab.failure.fanout.fanout5.saved; Ctrl+C to stop
2026-10-13T14:38:28.618+00:00 DURABLE RECEIVED id=7f036349-039f-46a9-a3c2-bfb898546490 body={"event_id": "7f036349-039f-46a9-a3c2-bfb898546490", "ord
er_id": "order-demo", "sequence": 1, "payload": {"text": "BEFORE"}}
2026-10-13T14:38:35.281+00:00 DURABLE RECEIVED id=fbbacbbf-1274-44e0-89f1-b4f049a6e167 body={"event_id": "fbbacbbf-1274-44e0-89f1-b4f049a6e167", "ord
er_id": "order-demo", "sequence": 1, "payload": {"text": "AFTER"}}
2026-10-13T14:38:36.296+00:00 Stopping consumer
^C
```

Durable consumer: receives BEFORE immediately on start (it was waiting in the queue) and later AFTER

ex2-4-temporary-consumer — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo2_late_subscriber.py temporary --run fanout5
2026-10-13T14:38:28.831+00:00 READY temporary queue=amq.gen-m9D5KsHAgp0kxh8I8Of6mg; Ctrl+C to stop
2026-10-13T14:38:35.280+00:00 TEMPORARY RECEIVED id=fbbacbbf-1274-44e0-89f1-b4f049a6e167 body={"event_id": "fbbacbbf-1274-44e0-89f1-b4f049a6e167", "o
rder_id": "order-demo", "sequence": 1, "payload": {"text": "AFTER"}}
2026-10-13T14:38:36.360+00:00 Stopping consumer
^C
```

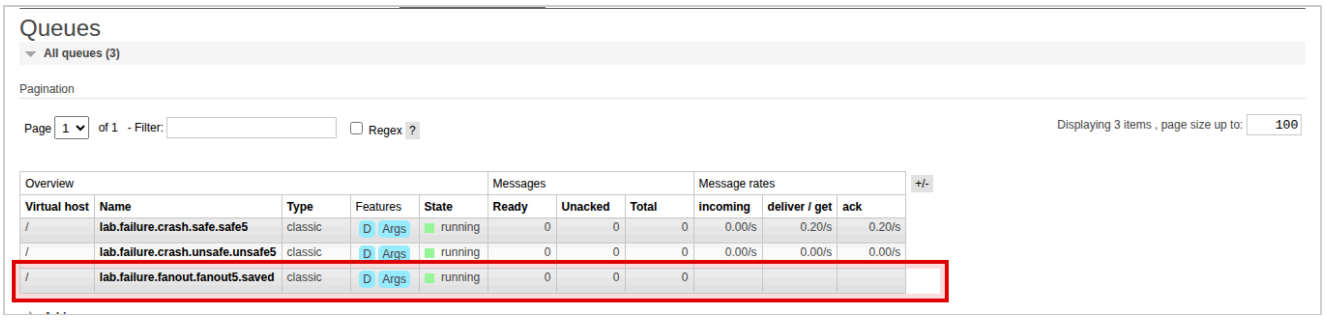
Temporary consumer: declares amq.gen-… after BEFORE was published and receives only AFTER

ex2-5-publish-after — bash

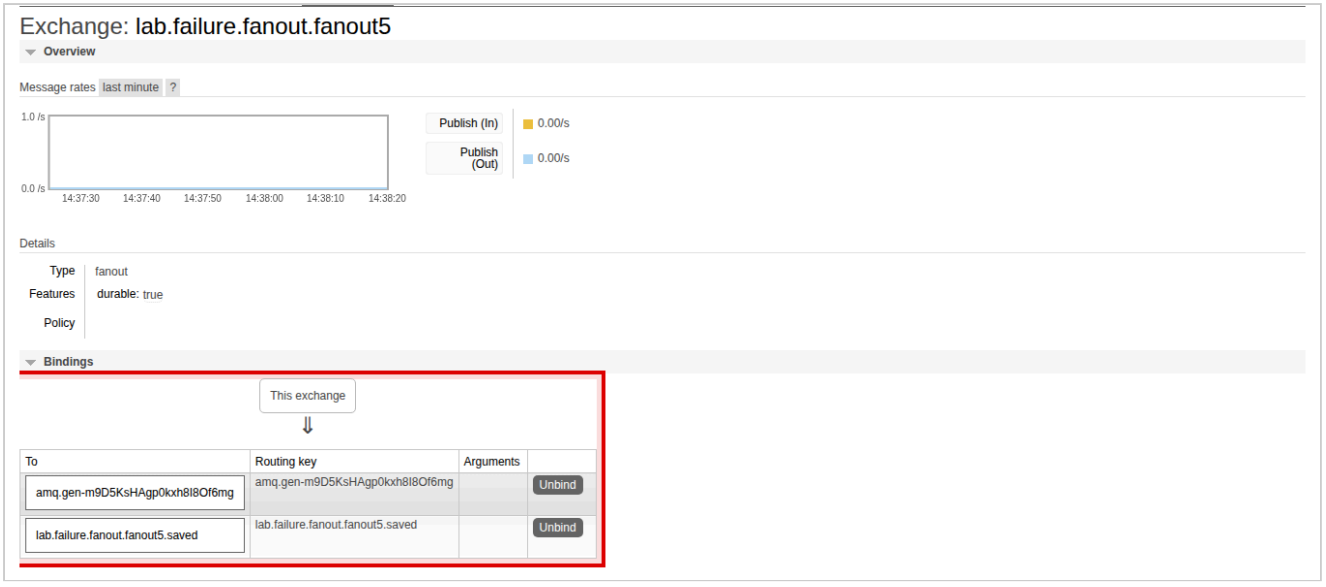
```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo2_late_subscriber.py publish --run fanout5 --message AFTER
2026-10-13T14:38:35.281+00:00 CONFIRMED id=fbbacbbf-1274-44e0-89f1-b4f049a6e167 payload={'text': 'AFTER'}
```

Publish AFTER while both consumers are attached

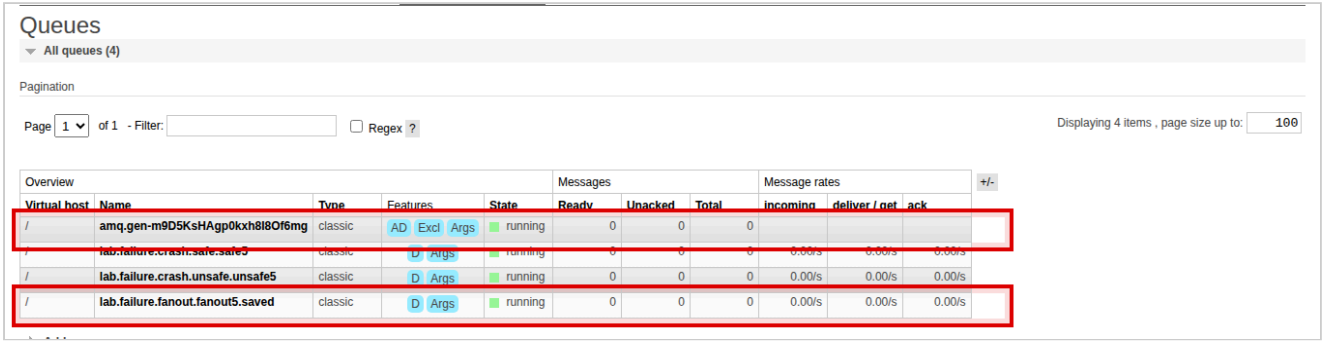
RabbitMQ management UI



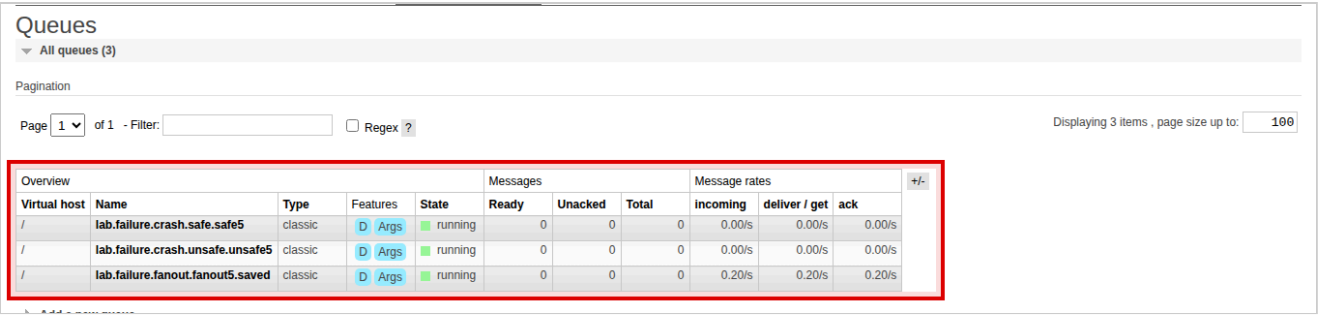
After BEFORE was published: the durable queue holds Ready=1 while no consumer exists; there is no temporary queue yet



Exchange bindings while both consumers run: the durable queue and the amq.gen-… exclusive queue are both bound to the fanout exchange



Queue list while both consumers run: the temporary queue shows the AD (auto-delete) and Excl (exclusive) features; the durable queue shows D



After both consumers were stopped: the exclusive queue was deleted with its connection; the durable queue remains

Results

Consumer	`BEFORE`	`AFTER`
Durable (`setup` ran first)	✅ received at 14:38:28.618 (on start-up)	✅ received at 14:38:35.281
Temporary (bound after `BEFORE`)	❌ never received	✅ received at 14:38:35.280

Explanation

A fanout exchange is a router, not a store: at the instant a message is published it is copied into every queue that is *currently* bound, and if nothing is bound the message is simply dropped. When `BEFORE` was published the only bound queue was the durable one, so the broker stored a copy there (Ready=1 in the screenshot) and the durable consumer received it the moment it attached, even though it had been offline during publication; buffering is a property of the queue, not of the exchange. The temporary queue did not exist yet, so there was nothing for the broker to copy `BEFORE` into, and when the exclusive queue was finally declared and bound it could only see traffic from that point on, which is why it got `AFTER` only. For the temporary consumer to receive `BEFORE`, its queue would have to be declared and bound before the publish happened, either by running it first or by making it a named, pre-declared queue like the durable one; nothing on the publisher side can fix this after the fact. Queue durability (the queue definition survives a broker restart) and message persistence (`delivery_mode=2`, the message is written to disk) are independent settings, and publishing to the exchange without ever running `setup` would deliver to zero queues with `mandatory=True` causing the publish to be returned as unroutable.

Exercise 3 — RabbitMQ: Delivery vs. Completion Order

Scenario. Three tasks are published in order A (5 s), B (1 s), C (2 s) to a work queue. First three workers with `prefetch_count=1` consume concurrently; then the same three tasks are consumed by a single worker.

Three-worker run — terminal output

ex3-parallel-publish — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py publish --run order5
2026-10-13T14:38:50.105+00:00 CONFIRMED id=f01305d0-5164-4f24-9379-cadfa6386ea9 payload={'task': 'A', 'seconds': 5, 'sequence': 1}
2026-10-13T14:38:50.106+00:00 CONFIRMED id=5c6ab3a6-6d85-4c26-ae83-56e502261c59 payload={'task': 'B', 'seconds': 1, 'sequence': 2}
2026-10-13T14:38:50.108+00:00 CONFIRMED id=7beeca2d-60fe-4e89-82f3-abec01552ff2 payload={'task': 'C', 'seconds': 2, 'sequence': 3}
```

Publisher: A, B, C confirmed in that order within 3 ms

ex3-parallel-W1 — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py worker --run order5 --name W1
2026-10-13T14:38:40.418+00:00 READY W1 queue=lab.failure.order.order5; Ctrl+C to stop
2026-10-13T14:38:50.106+00:00 W1 START B delay=1s redelivered=False
2026-10-13T14:38:51.108+00:00 W1 DONE B elapsed=1.00s
2026-10-13T14:38:55.900+00:00 Stopping consumer
^C
```

W1 took B and finished first at 14:38:51.108 (1.00 s)

ex3-parallel-W2 — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py worker --run order5 --name W2
2026-10-13T14:38:40.556+00:00 READY W2 queue=lab.failure.order.order5; Ctrl+C to stop
2026-10-13T14:38:50.107+00:00 W2 START C delay=2s redelivered=False
2026-10-13T14:38:52.109+00:00 W2 DONE C elapsed=2.00s
2026-10-13T14:38:55.932+00:00 Stopping consumer
^C
```

W2 took C and finished second at 14:38:52.109 (2.00 s)

ex3-parallel-W3 — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py worker --run order5 --name W3
2026-10-13T14:38:40.347+00:00 READY W3 queue=lab.failure.order.order5; Ctrl+C to stop
2026-10-13T14:38:50.104+00:00 W3 START A delay=5s redelivered=False
2026-10-13T14:38:55.106+00:00 W3 DONE A elapsed=5.00s
2026-10-13T14:38:55.967+00:00 Stopping consumer
^C
```

W3 took A and finished last at 14:38:55.106 (5.00 s)

One-worker run — terminal output

ex3-serial-publish — bash

```
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py publish --run serial5
2026-10-13T14:38:56.352+00:00 CONFIRMED id=347148b4-0350-4d0f-92d0-7dc604f32425 payload={'task': 'A', 'seconds': 5, 'sequence': 1}
2026-10-13T14:38:56.356+00:00 CONFIRMED id=c1d0a42d-2343-407d-b89f-1551e26762cb payload={'task': 'B', 'seconds': 1, 'sequence': 2}
2026-10-13T14:38:56.357+00:00 CONFIRMED id=0b6482f4-90be-4282-9ffc-10a696768d5b payload={'task': 'C', 'seconds': 2, 'sequence': 3}
```

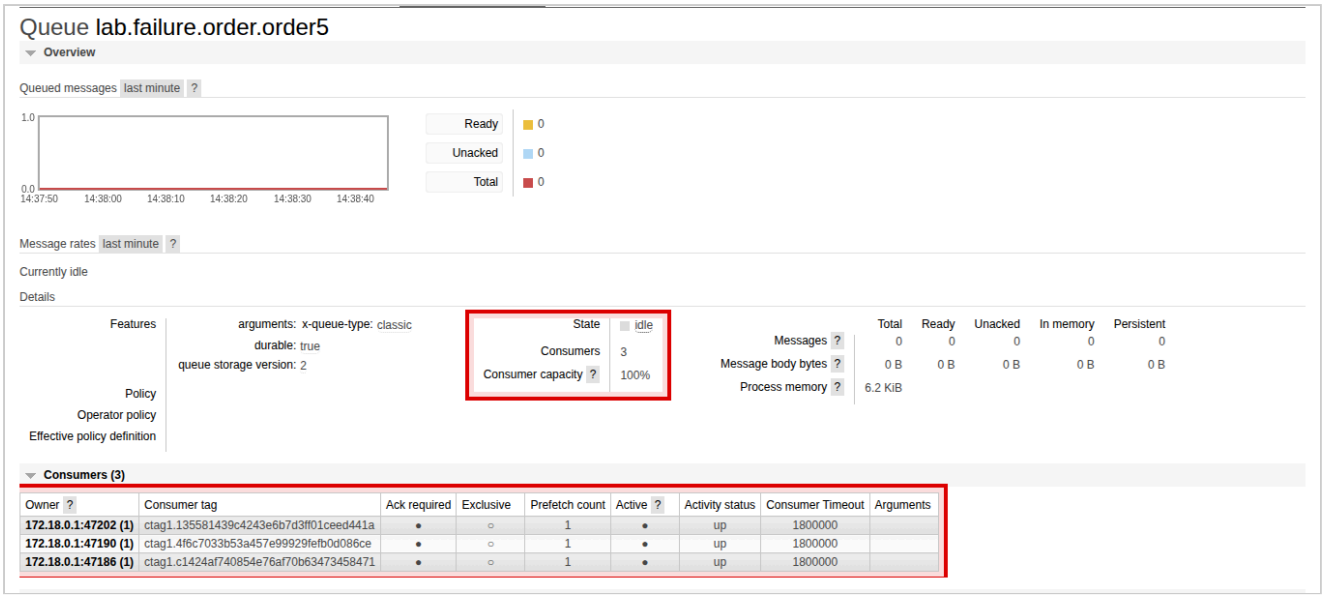
Publisher for the serial run

```
ex3-serial-ONLY — bash

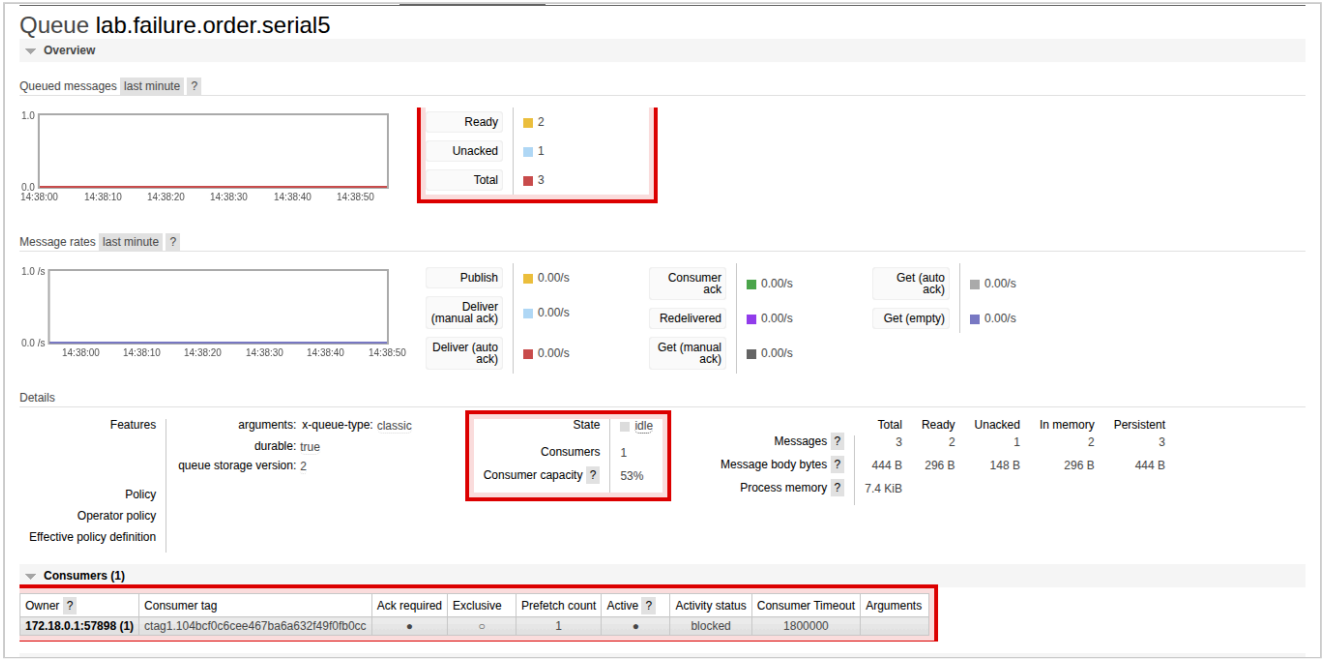
(.venv) @Watayut ~/messaging_lab_six_exercises $ python rabbitmq/demo3_completion_order.py worker --run serial5 --name ONLY
2026-10-13T14:38:56.152+00:00 READY ONLY queue=lab.failure.order.serial5; Ctrl+C to stop
2026-10-13T14:38:56.352+00:00 ONLY START A delay=5s redelivered=False
2026-10-13T14:39:01.359+00:00 ONLY DONE A elapsed=5.01s
2026-10-13T14:39:01.361+00:00 ONLY START B delay=1s redelivered=False
2026-10-13T14:39:02.362+00:00 ONLY DONE B elapsed=1.00s
2026-10-13T14:39:02.363+00:00 ONLY START C delay=2s redelivered=False
2026-10-13T14:39:04.366+00:00 ONLY DONE C elapsed=2.00s
2026-10-13T14:39:04.905+00:00 Stopping consumer
^C
```

Single worker: A, B, C are processed strictly one after another, finishing in delivery order after ≈8 s

RabbitMQ management UI



Before publishing: three consumers attached to the work queue, each with prefetch count 1 and manual ack



Serial run: one consumer with prefetch 1; the broker holds the remaining tasks and the consumer is shown as blocked while it works

Results

Workers	Delivery order	Completion order (from timestamps)	Elapsed from first START to last DONE
3 (parallel)	A → B → C	**B (14:38:51.108) → C (14:38:52.109) → A (14:38:55.106)**	≈ 5.0 s
1 (sequential)	A → B → C	**A (14:39:01.359) → B (14:39:02.362) → C (14:39:04.366)**	≈ 8.0 s

Explanation

RabbitMQ delivered A, B and C in the order they were published, but with three workers each task started at the same moment on a different process, so completion order was decided purely by task duration: B (1 s) finished first, C (2 s) second and A (5 s) last, and the whole batch took about 5 s, the length of the longest task. With `prefetch_count=1` the broker hands each worker at most one unacknowledged message at a time, which is what spread A, B and C across W3, W1 and W2 instead of letting the first worker grab all three; without it the first ready worker could have been sent all three messages and the others would have idled. `connection.sleep()` is used instead of `time.sleep()` so that pika keeps processing heartbeats and I/O during the simulated work; a plain sleep would block the client and could get the connection closed for missed heartbeats. The single-worker run guarantees A → B → C but costs the sum of all durations, about 8 s, because there is no concurrency at all. I would choose one worker (or, better, one worker *per ordering key*) only when the sequence of events genuinely matters, for example all events of the same order; for independent orders I would keep several workers and route each order's events to a single queue or partition so that different orders run concurrently while each order stays sequential.

Exercise 4 — Kafka: Consumer Groups and Partitions

> **About the `%3|…|FAIL|… Connect to ipv6#[::1]:19092 failed` lines** in the Kafka captures: in this Codespace `localhost` resolves to the IPv6 address `::1` first, but Docker only publishes the broker port on IPv4 `127.0.0.1`. librdkafka logs the failed IPv6 attempt and then connects on IPv4 within the same millisecond, so the lines are cosmetic; every publish was confirmed and every consumer received its records. A consumer capture ending in `exited with code -2` was simply terminated by Ctrl+C after it had written its summary.

Scenario. A 4-partition topic receives 40 keyed records (10 per partition) and is consumed by groups of 2, 4 and 5 consumers using the `range` assignor with manual offset commits. Finally a brand-new group `lab.ex4.analytics2` replays the first topic.

Partition assignment and records per consumer (from `evidence/ex4-*.jsonl`)

Group of 2 consumers — topic `lab.ex4.two2` (40 published: P0:10, P1:10, P2:10, P3:10; 40 consumed)

Consumer	Assigned partitions	Records completed	Per partition	`unique_completed`
C1	[0, 1]	20	P0:10, P1:10	20
C2	[2, 3]	20	P2:10, P3:10	20

Group of 4 consumers — topic `lab.ex4.four2` (40 published; 40 consumed)

Consumer	Assigned partitions	Records completed	Per partition	`unique_completed`
C1	[0]	10	P0:10	10
C2	[1]	10	P1:10	10
C3	[2]	10	P2:10	10
C4	[3]	10	P3:10	10

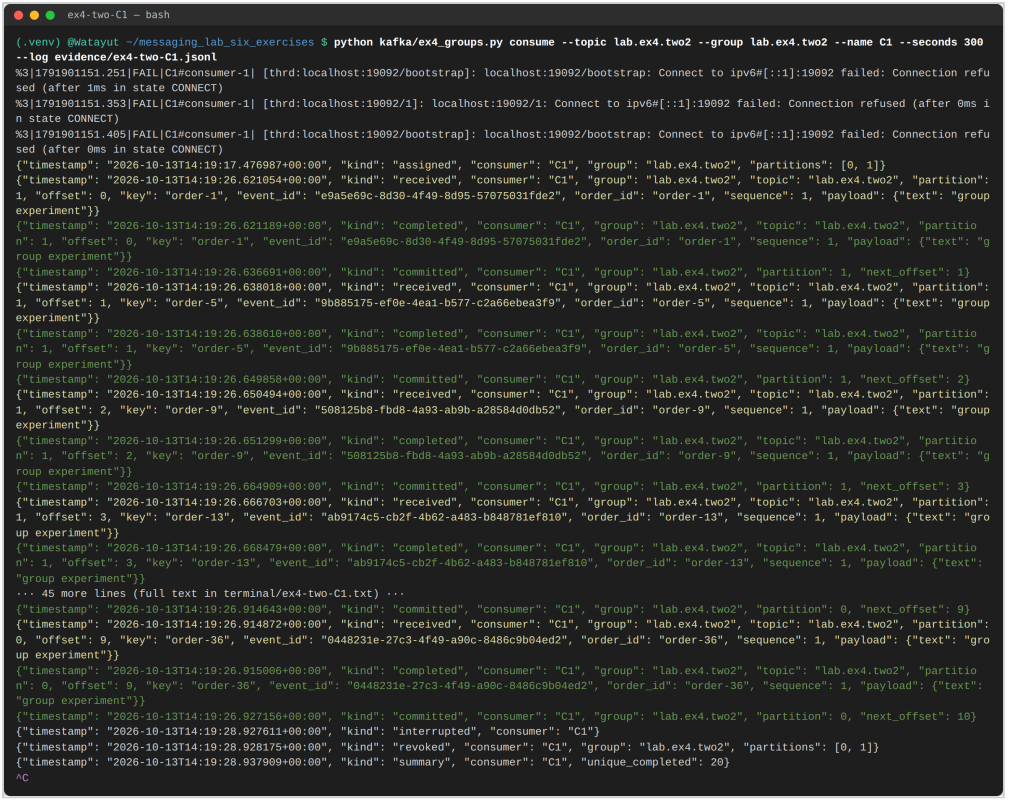
Group of 5 consumers — topic `lab.ex4.five2` (40 published; 40 consumed)

Consumer	Assigned partitions	Records completed	Per partition	`unique_completed`
C1	[0]	10	P0:10	10
C2	[1]	10	P1:10	10
C3	[2]	10	P2:10	10
C4	[3]	10	P3:10	10
C5	**[] (idle)**	**0**	–	0

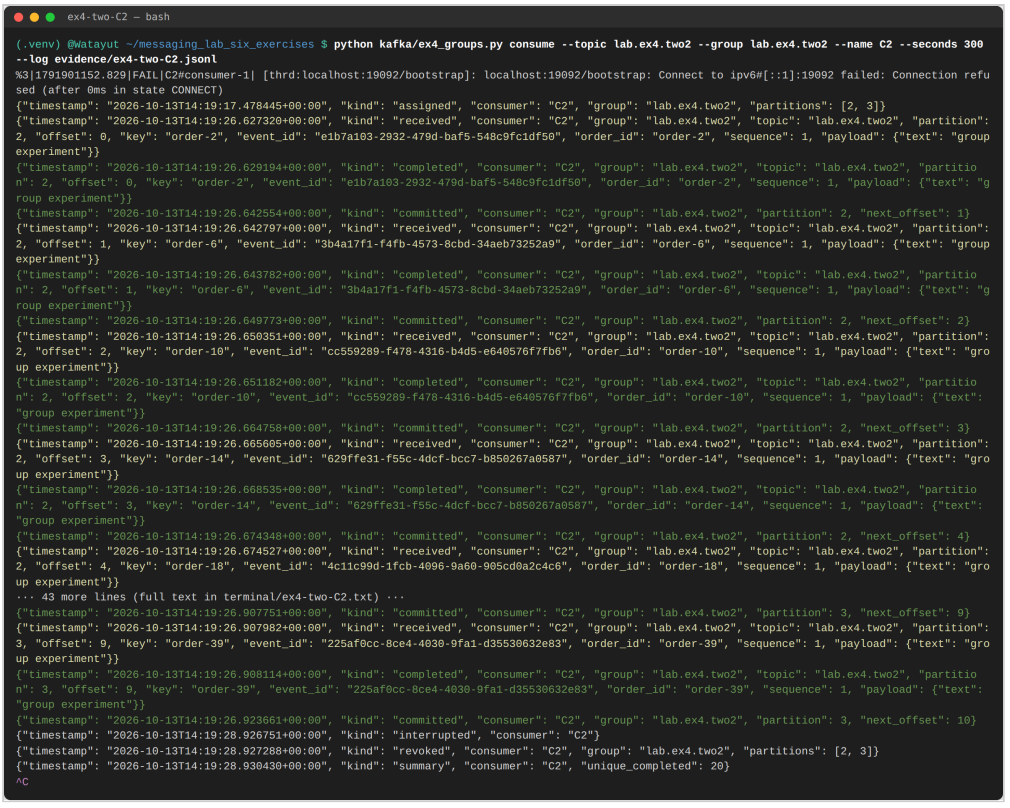
Analytics group replay — group `lab.ex4.analytics2` on `lab.ex4.two2`

Consumer	Assigned partitions	Records completed	Per partition	Offsets read per partition
Analytics	[0, 1, 2, 3]	**40**	P0:10, P1:10, P2:10, P3:10	0–9 on every partition

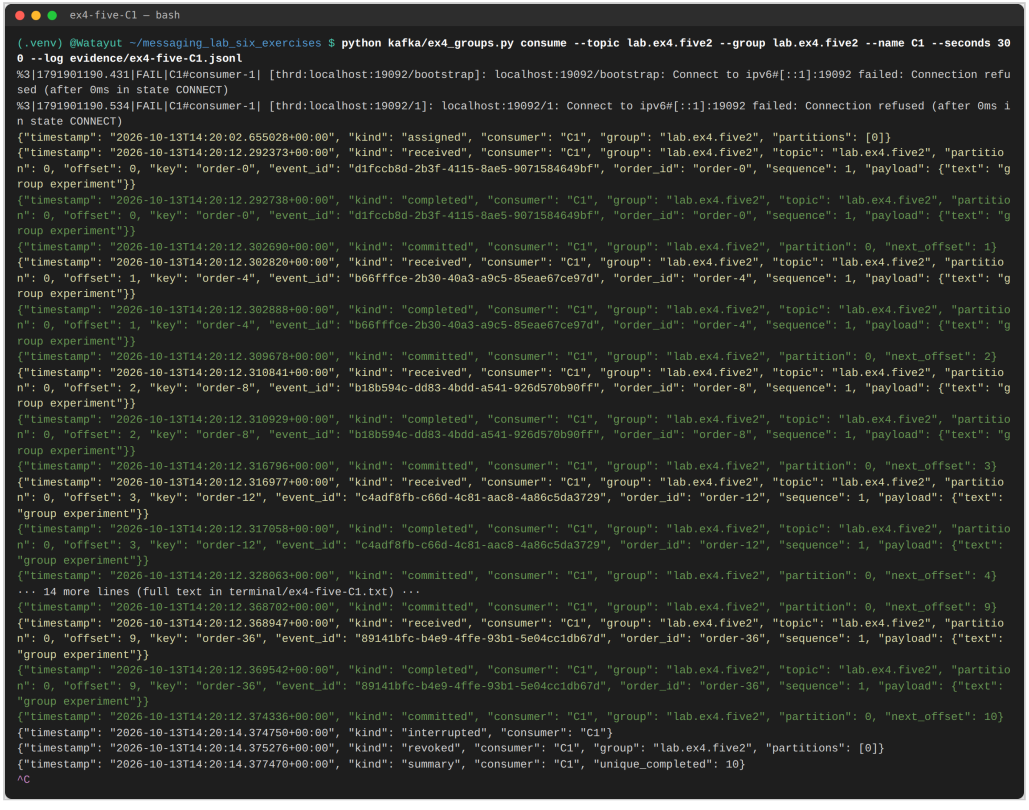
Terminal output (JSONL evidence as printed by the consumers)



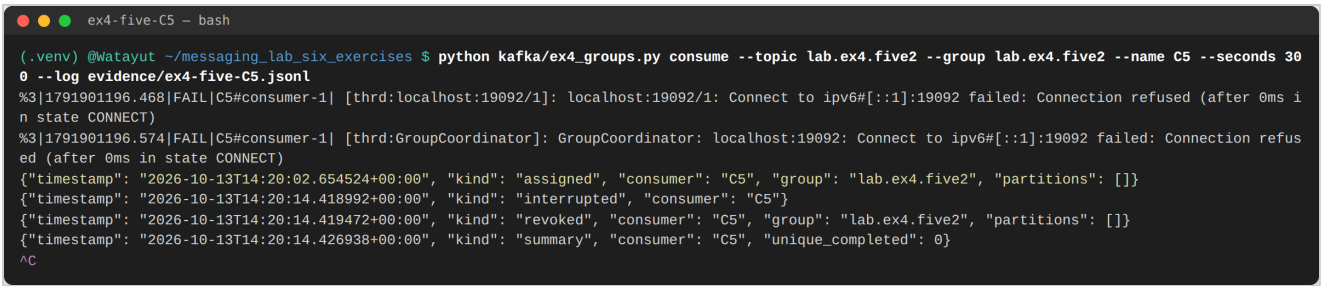
Group of 2, C1: assigned [0, 1], then received/completed/committed for 20 records



Group of 2, C2: assigned [2, 3], 20 records



Group of 5, C1: assigned [0] and 10 records



Group of 5, C5: assigned [] — it stays idle for the whole run and completes 0 records



Analytics group replay on lab.ex4.two2: a new group with no committed offset starts from earliest and reads all 40 records across all four partitions

Explanation

In Kafka the partition, not the message, is the unit of parallelism inside a consumer group: each partition is owned by exactly one consumer of the group at a time, so with 4 partitions the group of 2 got 2 partitions (20 records) each, the group of 4 got 1 partition (10 records) each, and in the group of 5 the fifth consumer, C5, was assigned an empty list and processed nothing because there was no partition left to give it. C5 would only get work if another member left: in my first run of this experiment, where the consumers were stopped one at a time, partition 3 was reassigned to C5 the moment C1 left, which shows that a rebalance redistributes ownership whenever a member joins or leaves (in the clean run shown here all five were stopped together, so C5 stayed idle throughout). The analytics group could read all 40 records that `lab.ex4.two2` had already delivered because offsets are tracked *per group*, so a group that has never committed anything starts from `auto.offset.reset=earliest`, and the records are still there because Kafka retains the log (here for 24 h) regardless of who has consumed it. This is the key difference from RabbitMQ's competing-consumer model, where a message is removed

from the queue once it is acknowledged and a late consumer can never see it; in Kafka consumption is a cursor over an immutable log, so adding independent readers is free. A consequence is that `auto.offset.reset` only matters the first time a group appears; re-running the analytics consumer with the same group id would resume from its committed offset 10 on each partition and receive nothing new.

Exercise 5 — Kafka: Keyed Ordering

Scenario. Three events per order (Created → Updated → Cancelled, sequence 1 → 2 → 3) are published to a 3-partition topic and consumed by three consumers in one group. In *keyed* mode the producer lets the default partitioner hash the key `order_id` (`partition=None`). In *broken* mode the producer forces each event of `order-123` onto a different partition (`partition=index`), and the consumers simulate 5 s / 1 s / 2 s of work for sequence 1 / 2 / 3.

Keyed mode — completion order from `evidence/ex5-keyed-*.jsonl`

Finish #	Order	Seq	Event	Partition	Offset	Consumer	Completed at (UTC)
1	order-123	1	Created	P1	0	C2	14:21:03.847
2	order-456	1	Created	P2	0	C3	14:21:03.850
3	order-789	1	Created	P0	0	C1	14:21:03.850
4	order-123	2	Updated	P1	1	C2	14:21:03.904
5	order-456	2	Updated	P2	1	C3	14:21:03.904
6	order-789	2	Updated	P0	1	C1	14:21:03.905
7	order-123	3	Cancelled	P1	2	C2	14:21:03.961
8	order-456	3	Cancelled	P2	2	C3	14:21:03.962
9	order-789	3	Cancelled	P0	2	C1	14:21:03.962

Order	Partition used	Completion sequence	In order?
order-123	P1 only	1 → 2 → 3	✅
order-456	P2 only	1 → 2 → 3	✅
order-789	P0 only	1 → 2 → 3	✅

Broken mode — completion order from `evidence/ex5-broken-*.jsonl`

Finish #	Order	Seq	Event	Partition	Offset	Consumer	Delay	Completed at (UTC)
1	order-123	2	Updated	P1	0	C2	1 s	14:21:25.386

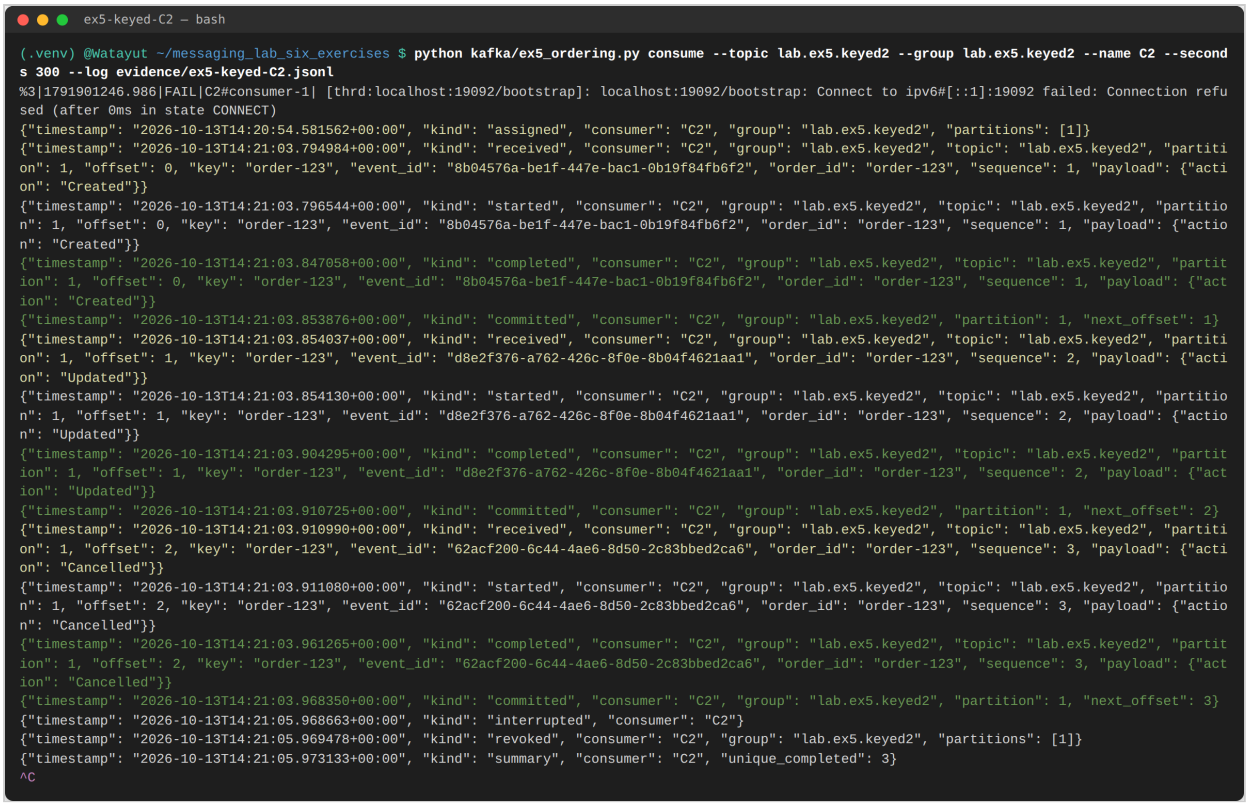
Finish #	Order	Seq	Event	Partition	Offset	Consumer	Delay	Completed at (UTC)
2	order-123	3	Cancelled	P2	0	C3	2 s	14:21:26.385
3	order-123	1	Created	P0	0	C1	5 s	14:21:29.385

Order	Partitions used	Completion sequence	In order?
order-123	P0, P1, P2	**2 → 3 → 1** (Updated, Cancelled, Created)	❌

Terminal output



Keyed publisher: nine records confirmed; the three events of each order landed on one partition with consecutive offsets 0, 1, 2



Keyed consumer C2 (partition 1): order-123 started and completed 1, 2, 3 in sequence



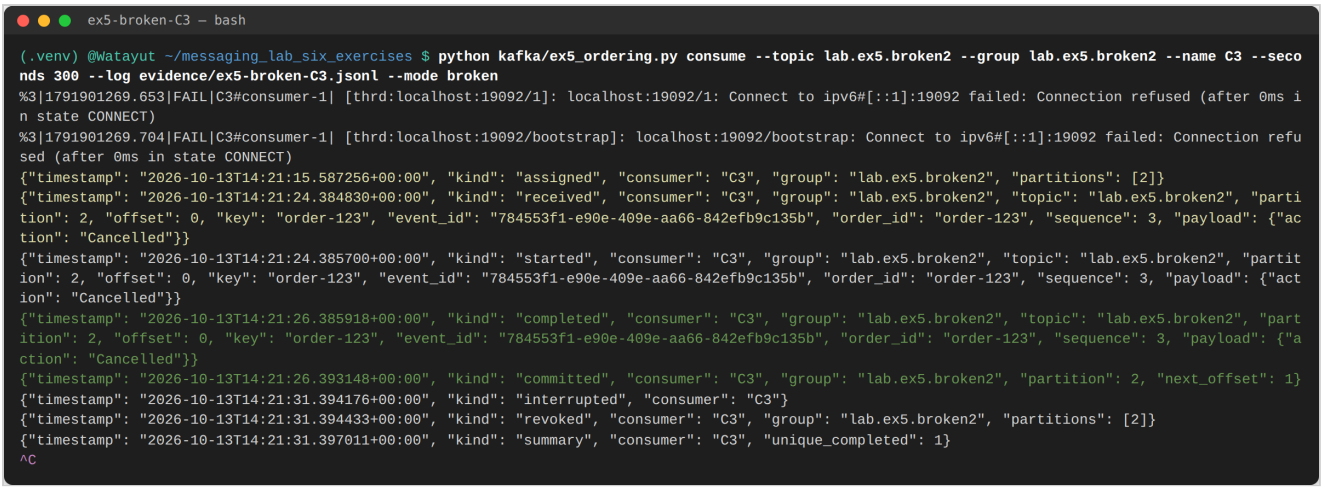
Broken publisher: the same order's events are forced to partitions 1, 2 and 0



Broken consumer C1 (partition 0): Created (seq 1, 5 s) finished last



Broken consumer C2 (partition 1): Updated (seq 2, 1 s) finished first



Broken consumer C3 (partition 2): Cancelled (seq 3, 2 s) finished second

The combined CSVs produced by `evidence_to_csv.py` are in `evidence/ex5-keyed.csv` (48 rows) and `evidence/ex5-broken.csv`.

Explanation

Kafka guarantees order only *within a partition*: records appended to one partition are read back in offset order by whichever single consumer owns it. In keyed mode the producer passed `partition=None` and the default partitioner hashed the key `order_id`, so all three events of `order-123` went to P1 at offsets 0, 1, 2 (and likewise order-456 to P2 and order-789 to P0), one consumer read each partition sequentially, and every order completed 1 → 2 → 3 even though three consumers were working in parallel on different orders. In broken mode the producer overrode the partitioner with `partition=index`, spreading one order over P0, P1 and P2; three different consumers then processed the three events concurrently and, because the simulated work took 5 s, 1 s and 2 s, the order finished Updated → Cancelled → Created, so a downstream system would have cancelled an order before it was created. The broker did nothing wrong in either case, it preserved exactly the per-partition order it was given; business-level ordering is the **producer's** responsibility (choose a key that captures the ordering requirement and never bypass it), while the consumer's responsibility is to process one partition sequentially and, defensively, to tolerate or detect out-of-sequence events. For an order-processing system I would key every event by `order_id`, keep producer idempotence and `acks=all` on, never set an explicit partition, and treat adding partitions as a migration

step because changing the partition count re-maps keys and can split an order's history across two partitions.

Exercise 6 — Kafka: Crash Before Offset Commit

Scenario. A one-partition topic receives one event. The consumer (`enable.auto.commit=False`, `enable.auto.offset.store=False`) applies the DB effect, then crashes with `os._exit(99)` *before* `commit(message=…, asynchronous=False)`. It is restarted with the same group id without publishing again, in *unsafe* and then in *safe* (idempotent) mode.

Terminal output

```
ex6-unsafe-2-consume-crash — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python kafka/ex6_crash_commit.py consume --topic lab.ex6.unsafe2 --group lab.ex6.unsafe2 --mode unsafe --crash-before-commit --log evidence/ex6-unsafe.jsonl
%3|1791901292.945|FAIL|ex6-worker#consumer-1| [thrd:localhost:19092/1]: localhost:19092/1: Connect to ipv6#[::1]:19092 failed: Connection refused (after 0ms in state CONNECT)
{"timestamp": "2026-10-13T14:21:36.061664+00:00", "kind": "assigned", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "partitions": [0]}
{"timestamp": "2026-10-13T14:21:36.169602+00:00", "kind": "received", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "topic": "lab.ex6.unsafe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f4371f0d-273b-4dee-99f0-8ece5b6392ea", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:36.176981+00:00", "kind": "db_committed", "applied": true, "business_effect_count": 1, "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "topic": "lab.ex6.unsafe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f4371f0d-273b-4dee-99f0-8ece5b6392ea", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:36.178189+00:00", "kind": "intentional_crash", "reason": "DB committed, Kafka offset NOT committed"}
[process exited with code 99]
```

Unsafe, first run: offset 0 received, DB committed with business_effect_count=1, intentional crash before the offset commit (exit 99)

```
ex6-unsafe-3-consume-restart — bash

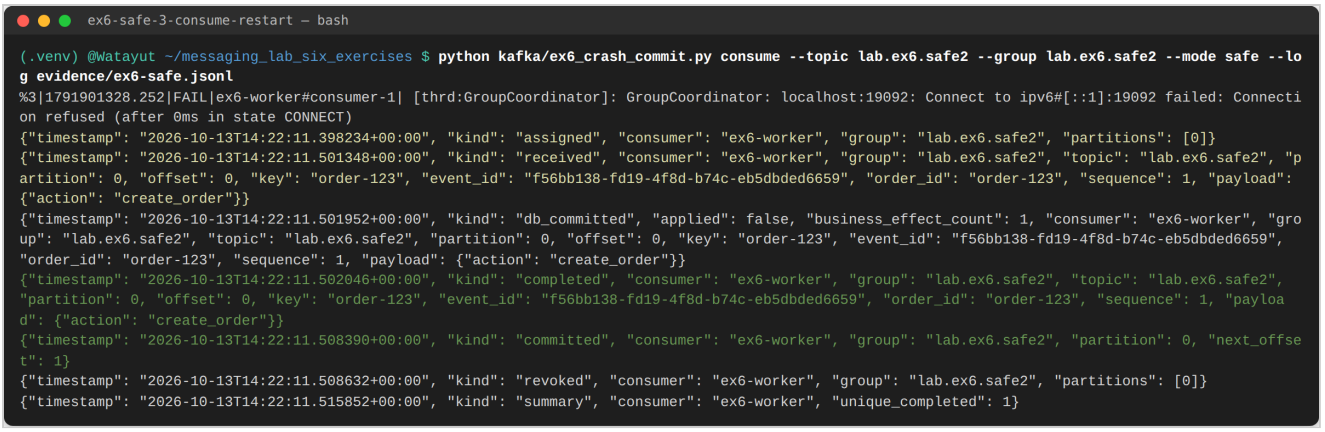
(.venv) @Watayut ~/messaging_lab_six_exercises $ python kafka/ex6_crash_commit.py consume --topic lab.ex6.unsafe2 --group lab.ex6.unsafe2 --mode unsafe --log evidence/ex6-unsafe.jsonl
{"timestamp": "2026-10-13T14:21:51.368470+00:00", "kind": "assigned", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "partitions": [0]}
{"timestamp": "2026-10-13T14:21:51.476332+00:00", "kind": "received", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "topic": "lab.ex6.unsafe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f4371f0d-273b-4dee-99f0-8ece5b6392ea", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:51.484951+00:00", "kind": "db_committed", "applied": true, "business_effect_count": 2, "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "topic": "lab.ex6.unsafe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f4371f0d-273b-4dee-99f0-8ece5b6392ea", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:51.485048+00:00", "kind": "completed", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "topic": "lab.ex6.unsafe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f4371f0d-273b-4dee-99f0-8ece5b6392ea", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:51.492843+00:00", "kind": "committed", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "partition": 0, "next_offset": 1}
{"timestamp": "2026-10-13T14:21:51.493140+00:00", "kind": "revoked", "consumer": "ex6-worker", "group": "lab.ex6.unsafe2", "partitions": [0]}
{"timestamp": "2026-10-13T14:21:51.501035+00:00", "kind": "summary", "consumer": "ex6-worker", "unique_completed": 1}
```

Unsafe, restart: offset 0 is replayed, applied=True again and business_effect_count=2, then committed next_offset=1

```
ex6-safe-2-consume-crash — bash

(.venv) @Watayut ~/messaging_lab_six_exercises $ python kafka/ex6_crash_commit.py consume --topic lab.ex6.safe2 --group lab.ex6.safe2 --mode safe --crash-before-commit --log evidence/ex6-safe.jsonl
%3|1791901312.869|FAIL|ex6-worker#consumer-1| [thrd:localhost:19092/1]: localhost:19092/1: Connect to ipv6#[::1]:19092 failed: Connection refused (after 0ms in state CONNECT)
{"timestamp": "2026-10-13T14:21:55.988205+00:00", "kind": "assigned", "consumer": "ex6-worker", "group": "lab.ex6.safe2", "partitions": [0]}
{"timestamp": "2026-10-13T14:21:56.092033+00:00", "kind": "received", "consumer": "ex6-worker", "group": "lab.ex6.safe2", "topic": "lab.ex6.safe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f56bb138-fd19-4f8d-b74c-eb5dbded6659", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:56.107947+00:00", "kind": "db_committed", "applied": true, "business_effect_count": 1, "consumer": "ex6-worker", "group": "lab.ex6.safe2", "topic": "lab.ex6.safe2", "partition": 0, "offset": 0, "key": "order-123", "event_id": "f56bb138-fd19-4f8d-b74c-eb5dbded6659", "order_id": "order-123", "sequence": 1, "payload": {"action": "create_order"}}
{"timestamp": "2026-10-13T14:21:56.108721+00:00", "kind": "intentional_crash", "reason": "DB committed, Kafka offset NOT committed"}
[process exited with code 99]
```

Safe, first run: applied=True, business_effect_count=1, crash before commit



Safe, restart: offset 0 replayed, applied=False, business_effect_count stays 1, offset committed

Results (from `evidence/ex6-unsafe.jsonl` and `evidence/ex6-safe.jsonl`)

Mode	Run	Offset received	`applied`	`business_effect_count`	Kafka offset committed?
Unsafe	first run (crash)	0	`True`	`1`	❌ no, crashed first
Unsafe	restart	**0 (replayed)**	`True`	**2** ← duplicate	✅ yes, `next_offset=1`
Safe	first run (crash)	0	`True`	`1`	❌ no, crashed first
Safe	restart	**0 (replayed)**	**`False`**	**1** ← deduplicated	✅ yes, `next_offset=1`

Explanation

This is the same failure mode as Exercise 1 with a different progress mechanism: in RabbitMQ progress is the per-message ACK and an unacknowledged message is requeued when the consumer's connection dies, whereas in Kafka progress is the committed offset of the group and a crash before the commit simply leaves the committed position at 0, so after the session timeout (10 s here) the restarted consumer is assigned partition 0 again and re-reads the same record. In both brokers the duplicate is therefore inevitable and the broker cannot know whether the database write happened; what differs is granularity (RabbitMQ tracks each message individually, Kafka tracks one cursor per partition and `commit(message=…)` records `offset+1`, the *next* record to fetch) and the fact that a Kafka record is never removed, it is only skipped by the cursor. The safe mode fixes both cases identically on the consumer side: the `INSERT OR IGNORE` dedupe marker and the business row share one SQLite transaction, so the replayed event is recognised by its `event_id`, `applied` is `False`, the count stays at 1, and only then is the offset committed. Both `enable.auto.commit=False` and `enable.auto.offset.store=False` are needed because with auto-store on, librdkafka would mark the record as "processed" the moment `poll()` returned it, and a later auto or manual commit could advance past an event that had not actually been handled, which is the symmetric and worse failure, silently skipping work. Committing synchronously

(`asynchronous=False`) makes the consumer wait for the broker to confirm the new position before it moves on, which is what we want after a side effect; asynchronous commits are cheaper and acceptable when a small amount of reprocessing after a crash is tolerable.

Overall Reflection

The result that surprised me most was Exercise 2: I had assumed that a *durable* fanout exchange would somehow hold the `BEFORE` message for late subscribers, but the exchange forgot it the instant it was routed, and the only reason the durable consumer saw it was that its queue had been bound a few seconds earlier; "durable" turned out to describe what survives a restart, not what is buffered. Exercise 5's broken mode was almost as instructive, because nothing failed, every record was delivered exactly once and in partition order, and the system still cancelled an order before creating it, which shows that correctness can be lost by a single wrong argument on the producer without any error being raised. Exercises 1 and 6 together made the at-least-once contract concrete: with either broker a crash between "effect committed" and "progress recorded" guarantees a redelivery, and I now see the broker's ACK or offset commit as a statement *about my own side effects*, so it must come last. The design principle I will apply is therefore: give every event a stable id, make the consumer idempotent by writing a dedupe marker in the same transaction as the business effect, acknowledge or commit only after that transaction succeeds, and choose the partition key (or queue) from the business ordering requirement rather than from throughput alone. Finally, I would make the "does a subscriber need history?" decision explicit in the architecture, using a Kafka-style log where independent replay matters and a RabbitMQ work queue where competing consumers and per-message acknowledgement are the better fit.